

Reflection

CSC322 Group E — CalcEngine

What I designed

I set out to build the part of a spreadsheet that nobody sees: a formula language, an expression tree, a dependency graph, and a recalculation loop, packaged as a .NET class library that the results portal can embed. What I finished is 382 tests over roughly six thousand lines of engine code, a Blazor grid that drives it, and two performance figures that sit two orders of magnitude inside the targets I was given.

The design turned out to rest on three decisions, and everything else followed from them.

The first was to make errors values, not exceptions. A malformed formula, a type error, a missing sheet, a division by zero and a circular reference are all things a spreadsheet *reports*; only a bug in the calling code is an exception. That single rule reorganised the API. `SetCellContent` returns a `CalculationResult` carrying changes, cycles and syntax errors rather than throwing; a cell whose formula does not parse keeps the text the user typed and shows `#PARSE!`, because an API cannot put up a dialogue box and a bulk importer needs the bad text preserved. The rule also decided the shape of `CellValue`: reading `AsNumber` on a text value *does* throw, because that is a caller's bug, and keeping the two kinds of failure apart is what stops "errors are values" from becoming "everything is a nullable and nobody checks".

The second was that the cost of an edit must follow the edit. This is the brief's 50 ms target, but it is really a structural claim: a 100,000-cell workbook must not do 100,000 units of work because one mark changed. Three things fell out of it. Cells became dense integer identifiers instead of addresses, so adjacency is array indexing. Traversal marks became generation-stamped arrays instead of hash sets, because clearing 100,000 flags on every keystroke costs more than the propagation those flags exist to support. And recalculation became a depth-first search from the edited cell over dependent edges, restricted to the affected subgraph, rather than a topological sort of the workbook. The measured result is 0.32 ms against a 50 ms budget. I am more pleased with the test than the number: `OnlyAffectedCellsAreRecomputed` asserts on how many cells were *evaluated*, not on their values, so a regression to the naive design fails a test rather than merely getting slower on a machine nobody is watching.

The third was that a range is not a set of cells. `=SUM(B2:B45)` looks like 44 dependencies and is not one, because the cell a lecturer fills in next term was empty when the formula was written and has no edge to expand into. That is not a performance argument; it is a correctness argument, and it is the one that sent us to a spatial index instead of an adjacency list. It is also the design decision I would most like to be asked about, because the reasoning is not obvious until you trace what happens when a student's absent mark finally arrives.

The two assigned features were the same lesson twice. Find and Replace is easy until you notice that "B2" occurs inside `=SUM(B2:B9)`, and that replacing it there rewrites every total on the sheet silently — which is why the engine can rewrite only inside a formula's *text literals*, using the parsed tree and the source span of each literal, and why it re-parses a rewritten formula before storing it. Duplicate

detection is easy until you notice that the number 5 and the text "5" hash to the same string, so an imported column of text marks appears to duplicate a typed one. Both features are one linear pass and one undoable command; the design effort went entirely into the cases where doing the obvious thing is wrong in a way nobody notices for a semester.

What I would do differently

I would build structural edits. The one place I stopped short is `RemoveDuplicates` in `ShiftUp` mode: it moves cell content up and does *not* rewrite the relative references inside moved formulas. I chose to refuse rather than to corrupt — the operation names the formulas it would move and declines unless the caller insists — and I still think refusing beats being silently wrong. But the honest reading is that insert-row, delete-row and delete-column, with the reference rewriting they imply, are a capability a calculation engine should have, and I scoped them out because they need their own visitor and their own test suite. The expression tree, the printer and the source spans are already the right foundation. Given another two weeks this is the first thing I would build.

I would benchmark the designs I rejected, not just the one I shipped. I rejected a whole-workbook recalculation on reasoning and never measured it. Our estimate is that it would have taken about 75 ms per edit — failing the target, but only on a slow machine, which is the most dangerous kind of failure. A five-minute experiment would have turned an argument into a number, and the number would have been worth having in the design portfolio.

I would write the performance tests earlier. They arrived in week five and immediately found a real defect: `Workbook` was consulting its constructor option rather than its mutable `AutomaticCalculation` property, so a workbook created for a bulk import could never be switched back to automatic calculation. Every unit test passed, because no unit test loaded a workbook the way a real client would. The benchmark harness was the first code that used the engine like a consumer, and it found the bug in its first run.

I would run the thing sooner, and by hand. The other two defects were both in the grid, and both came from the same line of code: `default(CellAddress)` is `A1` by design, so the component's "has the selection moved?" test was the wrong question to ask. First it was false on the very first render, so the edit buffer was never seeded and the first click elsewhere committed an empty string over `A1`, deleting the header of the sample sheet. Then — found later, by a user driving the GUI by hand rather than by any check of ours — the same test meant the selected cell never re-read the workbook when a `Replace All` rewrote it, so the grid showed stale text beside a formula bar showing the truth.

That second one is the more instructive. The engine was right, the tests were right, and the product was still wrong, because two views of the same cell were allowed to disagree and nothing in the suite compared them. Everything I know about the engine's internals came from tests; all three genuine bugs came from running it, and the last one only from running it the way a person would.

I would reconsider one API decision. `FunctionArguments` gives every function unevaluated arguments, which is what makes `IF` lazy and is right. But it makes every simple function slightly more verbose, and I papered over that with a `PointwiseFunction` adapter rather than offering a second,

simpler interface for the point-wise cases. A reviewer could fairly say I chose our convenience over our users'.

What the AI tools got wrong

I used AI assistants throughout, and the twenty entries in the [collaboration log](#) are the honest record. The pattern is consistent enough to state as a finding.

They are good at the shape of a solution and bad at the constraint that makes the problem hard. In one conversation, Claude proposed the correct five-box architecture I ended up building — and, a few messages later, a dependency graph that recalculated the entire workbook on every edit. Both answers were competent. The second missed the target by two orders of magnitude because the target was in our heads and not in the prompt. The same thing happened with duplicate keys (`string.Join("|", ...)`, which collides `5` with `"5"`), with `IF` (eager arguments, which breaks the exact guard `IF` exists for), and with range dependencies (expand into edges, which cannot fire for a cell that was empty when the formula was written).

They are confidently wrong about behaviour they have only read about. In one session I were told, correctly and usefully, about Excel's asymmetry between text read through a reference and text written into a formula — genuinely new to us, and it changed `SUM`. In the *same conversation* I were told that `COUNT` counts booleans inside ranges. It does not. I verified both in Excel before implementing either, and the habit of verifying is the only reason the first one is in the code and the second is not.

The critique exercise made the pattern legible. Asked for a "complete, production-quality" dependency-tracking module, ChatGPT produced code with the right data structure and the right algorithm, and with a representation invariant that breaks on every formula edit: `SetDependencies` rebuilds the forward map and never touches the reverse one, so a cycle the user has *fixed* is still reported. Four questions later, prompted specifically, it described exactly the design I had built. It had the knowledge. What it lacked was any reason to apply it. It optimised for the request as stated — short, clear, correct on a small example — which is what it should do, and precisely why the engineer owns the result.

Where they were genuinely valuable was in three places, all narrow. Reading a stack trace: our 20,000-cell cycle test died with `OutOfMemoryException` and the diagnosis — the cycle's error message was being interpolated once per member cell, and each message contains the whole path — came back correctly and immediately. Attacking a design once it exists: asked to find holes in our block index, it produced the unbounded-range case that became the fallback list. And recalling documented behaviour I did not know to look for, subject to verification.

The habit I ended with is narrower than "use AI carefully". It is: *state the constraint in the prompt, and then test the constraint rather than the output*. Every AI-suggested design that got into this repository is guarded by a test that would fail if someone replaced it with the plausible alternative — not a test that the answer is right, but a test that the *reason* still holds. `OnlyAffectedCellsAreRecomputed` counting evaluations, `If_DoesNotEvaluateTheBranchItDoesNotTake`, `ANumberIsNotTheSameAsTheTextThatLooksLikeIt`,

`WritingIntoAPreviouslyEmptyCellOfARangeStillTriggersIt`. Each of those exists because a confident, plausible, wrong answer was on the table and I could name what it would break.

That is the part of this project I expect to still be using in five years.